PROYECTO INGENIERÍA DE SOFTWARE 2

ENTREGA NÚMERO 1 - INVESTIGACIÓN Y FUNDAMENTACIÓN

Presentado por:

Santiago Lopez

Oswin Cuaran



Universidad Nacional de Colombia

Facultad de Administración

Administración de Sistemas Informáticos

Segundo semestre del 2026

Introducción

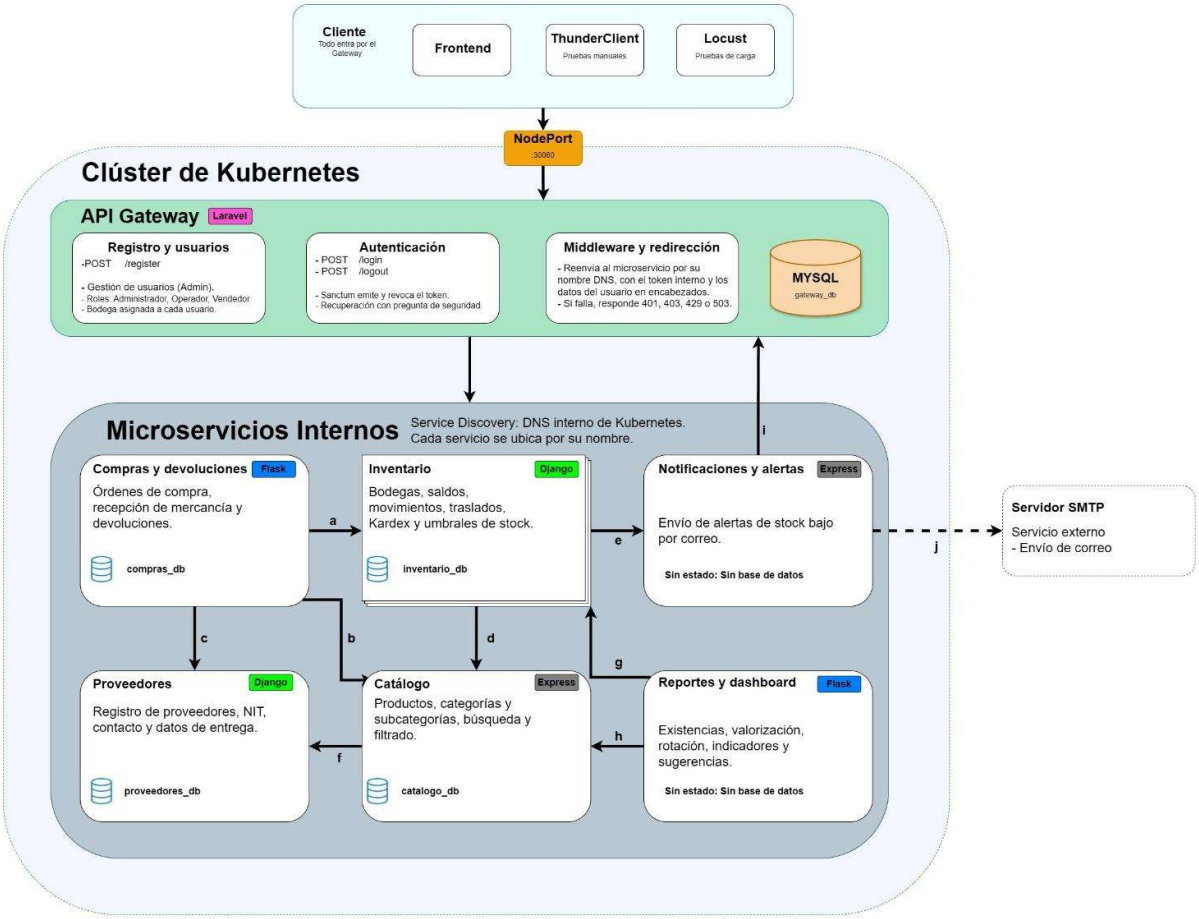
Esta segunda entrega corresponde a la fase de diseño. En ella presentamos el diagrama de arquitectura, donde un API Gateway propio en Laravel funciona como única entrada al sistema y se encarga del registro, la autenticación con Sanctum y la redirección de peticiones. Detrás de él hay seis microservicios construidos con Django, Express y Flask, que se despliegan en Kubernetes y se comunican por medio de API REST. También presentamos los modelos entidad relación de las cinco bases de datos del sistema, normalizados hasta la tercera forma normal.

Arquitectura de microservicios

Las peticiones de los clientes entran al clúster de Kubernetes por un NodePort y llegan al API Gateway, desarrollado en Laravel. El Gateway es la única entrada al sistema. Se encarga del registro de usuarios, la autenticación con Sanctum y la redirección de cada petición al microservicio que corresponde, y guarda los usuarios y los tokens en su propia base de datos.

Detrás del Gateway están los seis microservicios internos. Compras, Inventario, Proveedores y Catálogo tienen cada uno su propia base de datos MySQL. Notificaciones y Reportes son servicios sin estado, por lo que no guardan información y la consultan a los demás servicios cuando la necesitan. Los servicios se comunican entre sí por medio de API REST y se ubican por su nombre gracias al DNS interno de Kubernetes. Notificaciones es el único que se conecta con un servicio externo, un servidor SMTP para enviar los correos de alerta.

Arquitectura de microservicios



Relaciones entre microservicios

a. Compras ⟶ Inventario ⟶ Registra las entradas por recepción y devoluciones.
b. Compras ⟶ Catálogo ⟶ Valida los productos de la orden.
c. Compras ⟶ Proveedores ⟶ Valida el proveedor de la orden.
d. Inventario ⟶ Catálogo ⟶ Valida que el producto exista y esté activo.
e. Inventario ⟶ Notificaciones ⟶ Avisa que el stock bajó para enviar correo.
f. Catálogo ⟶ Proveedores ⟶ Valida el proveedor del producto.
g. Reportes ⟶ Inventario ⟶ Consulta saldos y ventas del periodo para calcular la rotación.
h. Reportes ⟶ Catálogo ⟶ Consulta precios y categorías para calcular la valorización.
i. Notificaciones ⟶ Gateway ⟶ Pide los destinatarios de la alerta según la bodega.
j. Notificaciones ⟶ SMTP ⟶ Envía el correo de alerta.

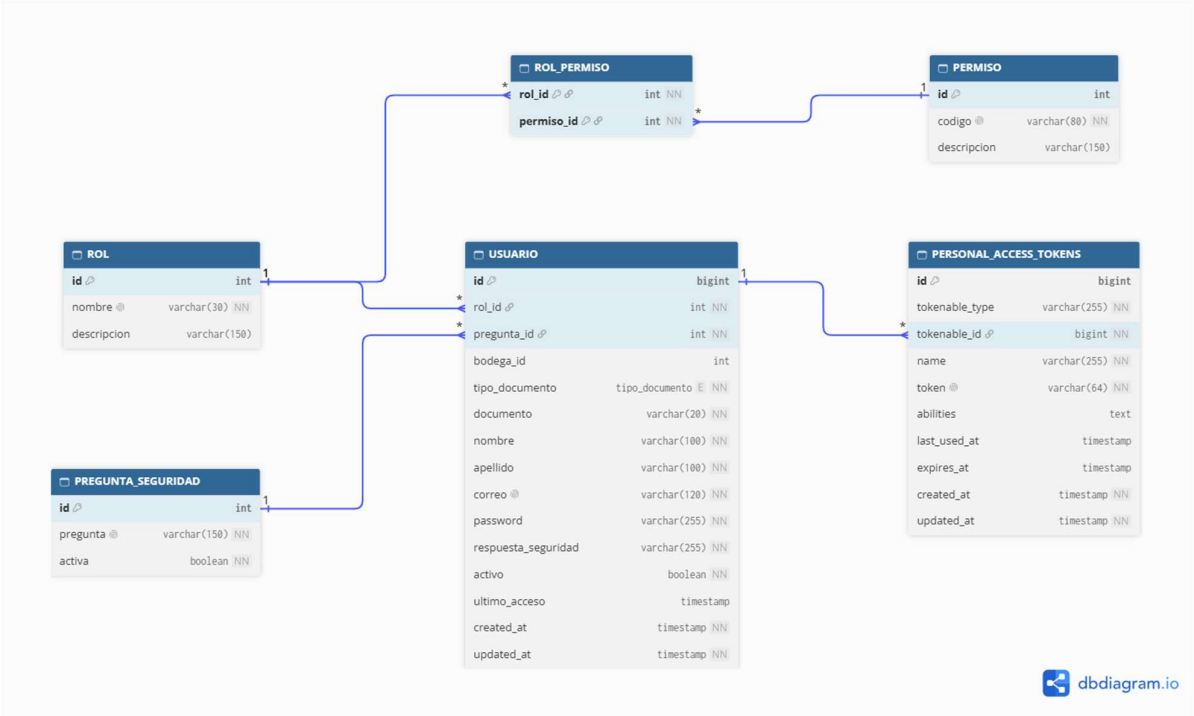
FUENTE: ELABORACIÓN PROPIA

Modelos Relacionales de cada microservicio

- **API Gateway**

  La base de datos gateway_db pertenece al API Gateway y guarda la información de los usuarios del sistema. La tabla USUARIO almacena los datos de cada empleado, su rol, la bodega a la que pertenece y la pregunta de seguridad para recuperar su contraseña. Los roles

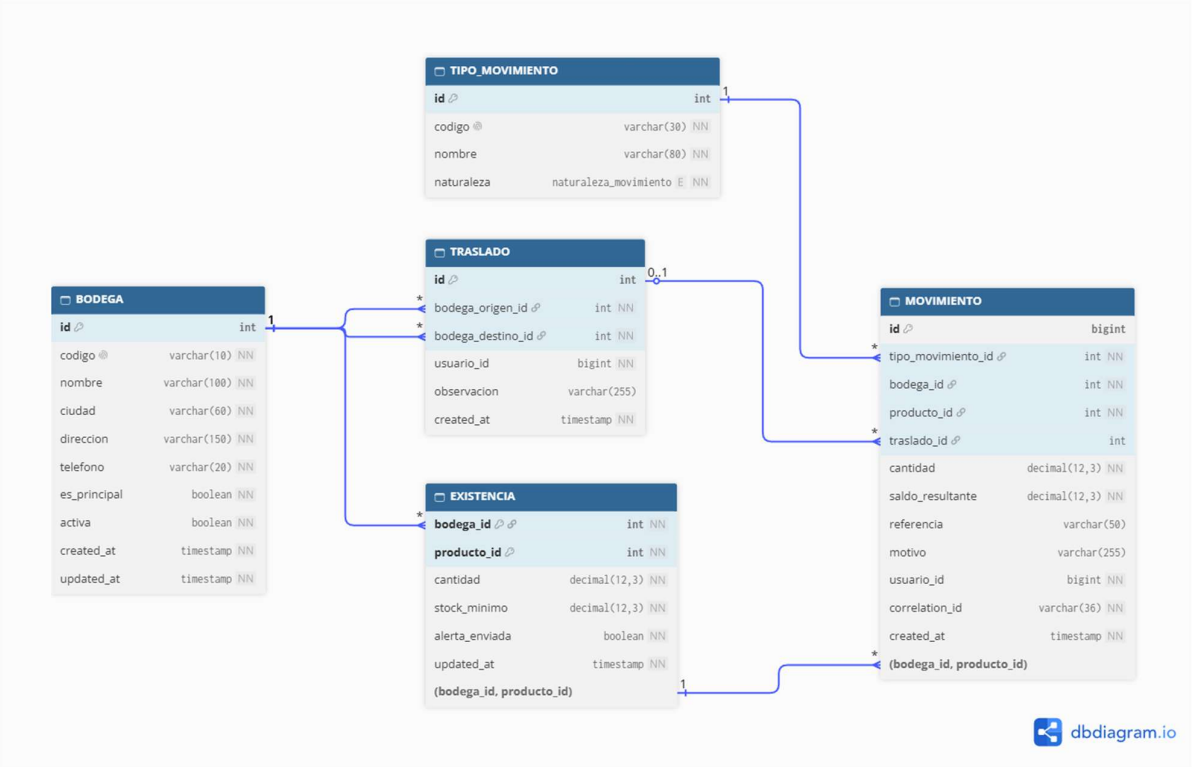
se relacionan con sus permisos por medio de ROL_PERMISO, una entidad débil que resuelve la relación de muchos a muchos entre ROL y PERMISO. La tabla PERSONAL_ACCESS_TOKENS es propia de Sanctum y guarda los tokens que se entregan al iniciar sesión. Las contraseñas y las respuestas de seguridad se guardan con hash, nunca en texto plano.



FUENTE: ELABORACIÓN PROPIA

- **Inventario**

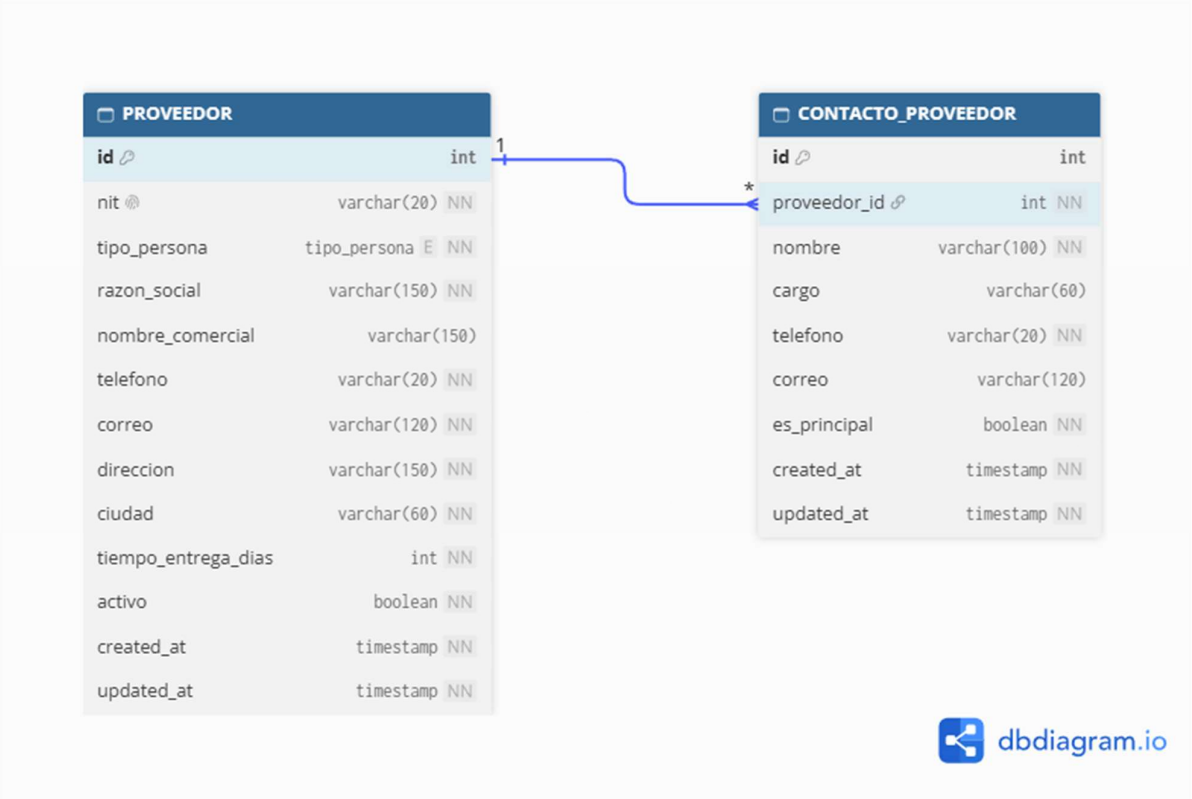
  La base de datos inventario_db guarda las bodegas de la empresa y todo lo que entra y sale de ellas. La tabla EXISTENCIA guarda la cantidad de cada producto en cada bodega junto con su stock mínimo, y es una entidad débil de BODEGA porque se identifica con la bodega y el producto. La tabla MOVIMIENTO funciona como kardex, registra cada entrada, salida, ajuste o traslado con su usuario y su fecha, y sus registros nunca se modifican. TIPO_MOVIMIENTO indica si cada movimiento suma o resta al saldo, y TRASLADO agrupa los movimientos que pasan producto de una bodega a otra. El campo producto_id hace referencia a Catálogo, pero no es llave foránea porque ese dato vive en otra base de datos.



FUENTE: ELABORACIÓN PROPIA

- **Proveedores**

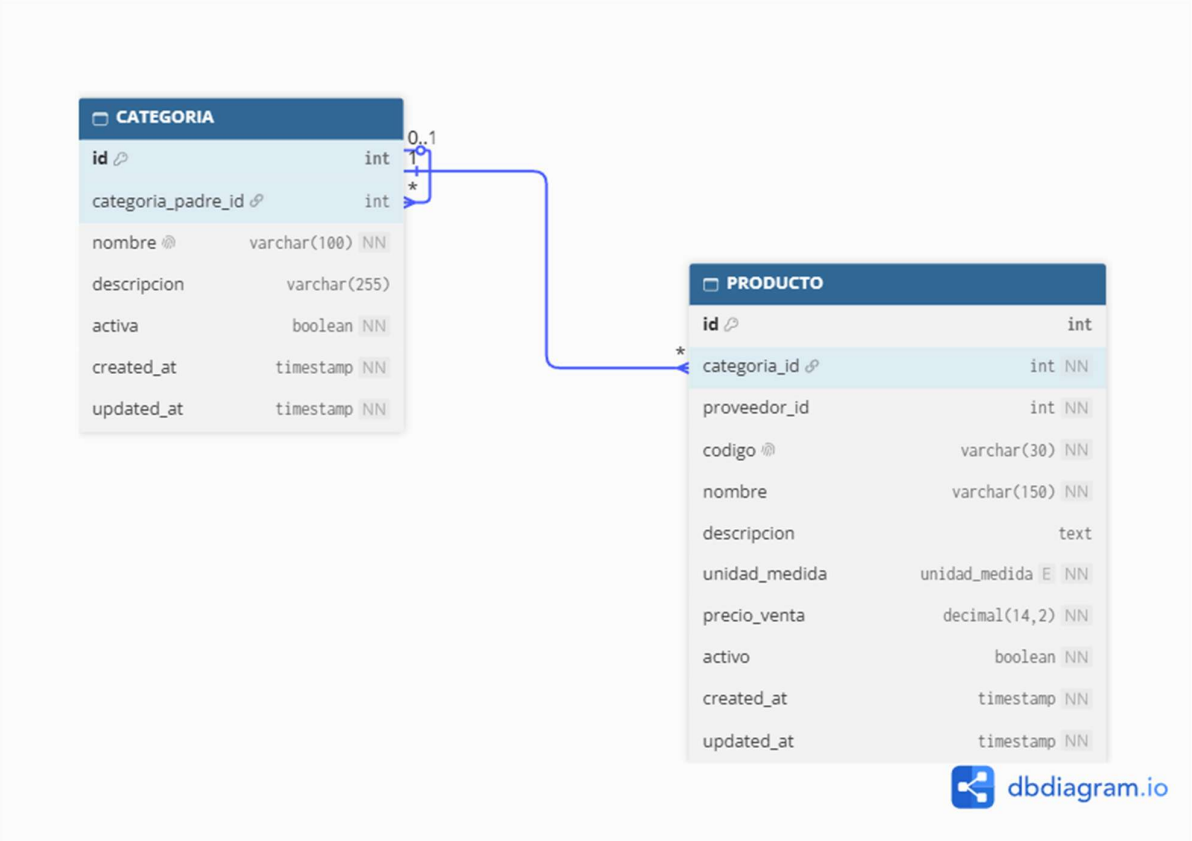
  La base de datos proveedores_db guarda los proveedores a los que la ferretería les compra. La tabla PROVEEDOR almacena el NIT, la razón social, los datos de contacto, la ciudad y el tiempo de entrega de cada uno. Como un proveedor puede tener varias personas de contacto, estas se guardan aparte en la tabla CONTACTO_PROVEEDOR. Los proveedores no se borran, solo se desactivan para conservar su historial.



FUENTE: ELABORACIÓN PROPIA

- **Catálogo**

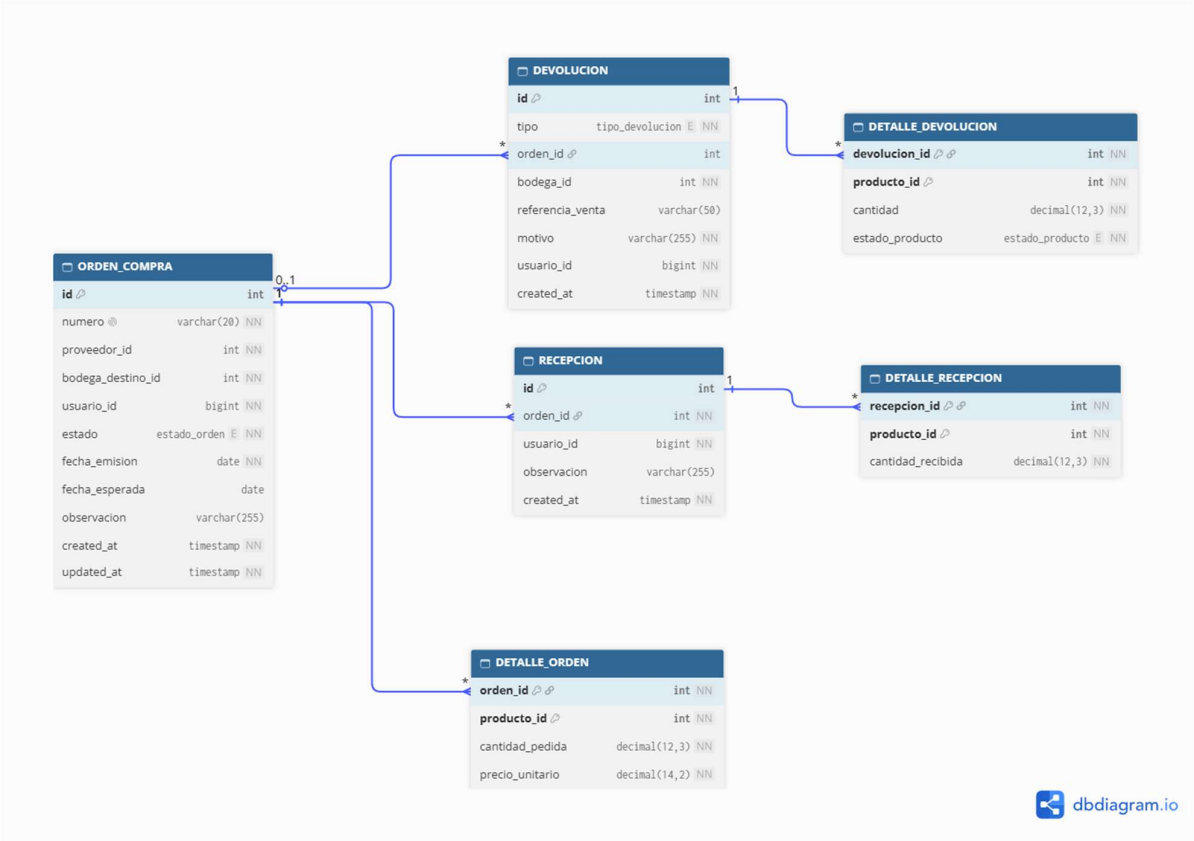
  La base de datos catalogo_db guarda los productos que vende la ferretería y sus categorías. La tabla CATEGORIA tiene una relación consigo misma, lo que permite crear subcategorías dentro de cada categoría. La tabla PRODUCTO guarda el código, el nombre, la unidad de medida, el precio de venta y la categoría de cada producto. El campo proveedor_id indica quién surte el producto y hace referencia a la base de datos de Proveedores.



FUENTE: ELABORACIÓN PROPIA

- **Compras y devoluciones**

  La base de datos compras_db guarda las órdenes de compra, la recepción de la mercancía y las devoluciones. Cada ORDEN_COMPRA tiene su DETALLE_ORDEN con los productos, las cantidades y los precios pactados. Una orden puede llegar en varias partes, por eso cada RECEPCION tiene su propio detalle con lo que realmente se recibió. Las devoluciones pueden ser de clientes o a proveedores, y su detalle indica si el producto volvió en buen estado. Los tres detalles son entidades débiles, porque se identifican con su encabezado y el producto.



FUENTE: ELABORACIÓN PROPIA

Contratos API

Cada microservicio expone sus funciones por medio de una API REST documentada con OpenAPI. El contrato define los endpoints, los datos que recibe cada uno, las respuestas y los errores posibles. Todas las peticiones externas pasan por el Gateway con el token de Sanctum, y los errores siguen un mismo formato con un código, un mensaje y el Correlation ID de la petición. Como ejemplo se incluye el contrato completo de Inventario en el repositorio, y en la siguiente tabla se resumen los endpoints principales de cada servicio.

Servicio	Endpoints principales
API Gateway	POST /register, POST /login, POST /logout, POST /password/recuperar, GET, POST y PUT /usuarios
Inventario	GET y POST /bodegas, POST /movimientos/salidas, GET /movimientos, POST /traslados, GET /stock/{productoId}, GET /existencias/bajo-minimo
Proveedores	GET y POST /proveedores, PUT /proveedores/{id}, PATCH

	/proveedores/{id}/baja, GET y POST /proveedores/{id}/contactos
Catálogo	GET y POST /productos, PUT /productos/{id}, PATCH /productos/{id}/baja, GET y POST /categorias, PUT y DELETE /categorias/{id}
Compras y devoluciones	GET y POST /ordenes, PATCH /ordenes/{id}/estado, POST /ordenes/{id}/recepciones, GET y POST /devoluciones
Notificaciones	POST /alertas, de uso interno
Reportes	GET /reportes/existencias, GET /reportes/valorizacion, GET /reportes/rotacion, GET /dashboard

FUENTE: ELABORACIÓN PROPIA

Estrategia de contenerización y orquestación

Cada componente del sistema se empaqueta en su propia imagen de Docker, que incluye solo su código y sus dependencias. El Gateway usa la imagen oficial de PHP con Laravel, los servicios en Django y Flask usan la imagen oficial de Python, y los servicios en Express usan la de Node.js. Cada una de las cinco bases de datos corre en su propio contenedor con la imagen oficial de MySQL. Ninguna imagen guarda contraseñas ni direcciones dentro del código. Esos datos se entregan como variables de entorno al momento de desplegar, así el mismo contenedor funciona igual en cualquier computador. Docker Compose solo se usará como apoyo para probar en el entorno local.

El despliegue se hace en un clúster de Kubernetes, dentro de un espacio de nombres llamado ferreteria-cafetera. Cada microservicio tiene un Deployment, que mantiene sus contenedores en ejecución y los reinicia si fallan, y un Service de tipo ClusterIP, que lo hace accesible sólo dentro del clúster por su nombre. El Gateway es el único con un Service de tipo NodePort, en el puerto 30080, por donde entran todas las peticiones externas. Las bases de datos se despliegan como StatefulSet con un volumen persistente, para que la información no se pierda cuando un contenedor se reinicia.

La configuración se separa del código. Las direcciones internas de los servicios y los nombres de las bases de datos van en un ConfigMap, mientras que las contraseñas, el token interno y las credenciales del correo van en un Secret. Para atender los picos de demanda, el Gateway e Inventario se escalan automáticamente con un HorizontalPodAutoscaler según el uso de procesador. Notificaciones y Reportes no tienen base de datos, por lo que también pueden escalar fácilmente si se necesita.

Componente	Imagen base	Objeto en Kubernetes	Service	Réplicas
API Gateway	PHP con Laravel	Deployment	NodePort 30080	2, con escalado automático
Inventario	Python con Django	Deployment	ClusterIP 8000	2, con escalado automático
Proveedores	Python con Django	Deployment	ClusterIP 8000	1
Catálogo	Node.js con Express	Deployment	ClusterIP 3000	1
Compras y devoluciones	Python con Flask	Deployment	ClusterIP 5000	1
Notificaciones	Node.js con Express	Deployment	ClusterIP 3000	1
Reportes	Python con Flask	Deployment	ClusterIP 5000	1
Cinco bases de datos	MySQL 8	StatefulSet con volumen	ClusterIP 3306	1 cada una

FUENTE: ELABORACIÓN PROPIA